



Figure 2: **Log Probability Estimation in *diffu*-GRPO.** After generating completion o from prompt q using full diffusion denoising (left), we compute token-level log probabilities with a single forward pass per masking pattern (mid) and use the log-probability of one-step unmasking as our estimation. During each policy gradient update, we apply a random masking pattern to the prompt, creating q' , while keeping the completion fully masked (right). The gradient of colors in the per-token log probabilities demonstrates that each distinct masking pattern yields a different estimate of the per-token log probabilities. This serves as a form of regularization for policy optimization, allowing more gradient updates per batch and thereby reducing the number of online generations needed for RL training.

as $D_{\text{KL}}[\pi_{\theta}(\cdot|q) \parallel \pi_{\text{ref}}(\cdot|q)] = \mathbb{E}_{o \sim \pi_{\theta}(\cdot|q)} \left[\sum_{k=1}^{|o|} \log \frac{\pi_{\theta}(o^k | q, o^{<k})}{\pi_{\text{ref}}(o^k | q, o^{<k})} \right]$. Unlike AR models, dLLMs do not adhere to sequential factorization of the sequence log-probability. Meanwhile, the per-token log-probability are also costly to compute since the decoding process invokes the unmasking predictor f_{θ} multiple times³. As the first step, we propose an efficient log-probability estimator in Section 3.1. Next, using these estimators, we introduce *diffu*-GRPO, a variant of GRPO for dLLMs in Section 3.2. Last, we discuss our SFT recipe in Section 3.3.

3.1 Efficient Log Probability Estimation for Masked dLLMs

For sequence log-probability, we use a mean-field approximation that decomposes it into a product of independent per-token log-probabilities. For per-token log-probability, we introduce an estimation method that only calls f_{θ} once.

Mean-Field Approximation of Sequence Log Probability. As opposed to AR models, dLLMs treat the token sequence as a whole and therefore its sequence-level log-probability lacks the AR decomposition. To efficiently estimate it, we use a simple mean-field decomposition to approximate $\log \pi_{\theta}(o|q)$ by $\sum_{k=1}^{|o|} \log \pi_{\theta}(o^k|q)$. The per-token log-probability estimation is introduced below.

One-Step Per-Token Log Probability Estimation with Prompt Masking. Let $\oplus$ denote the concatenation operator. Given a prompt q , the decoding process starts from an initial sequence $q \oplus \text{mask} \oplus \dots \oplus \text{mask}$ (up to a preset length). To compute the log-probability of o , we perturb q where every token is randomly masked out with probability p_{mask} , resulting in a new prompt q' . We then do one-step unmasking to obtain $\log f_{\theta}(o^k | q' \oplus \text{mask} \dots \oplus \text{mask})$ and use it as an estimation of $\log \pi_{\theta}(o^k | q)$, $1 \leq k \leq |o|$. We discuss the motivation of using a masked prompt q' in the next section.

We note that LLaDA [30, Algorithm 3] uses a Monte Carlo type of approximation to estimate the log-probabilities, where they use a MC sample size is 128. This estimator is inefficient for online RL, since it creates a large computational graph with hundreds of forward passes, resulting in inefficient policy optimization and excessive memory usage.

3.2 *diffu*-GRPO: Policy Gradient Optimization for Masked dLLMs

Using the log-probability estimators proposed in Section 3.1, we extend GRPO to masked dLLMs. Note that our estimation technique is broadly applicable and can readily extend to other policy gradient methods such as PPO [37] or REINFORCE [44].

³In other words, π_{θ} is a composition of M f_{θ} functions for a M -step decoding process

Algorithm 1 *diffu*-GRPO: Policy Gradient Optimization for Masked dLLMs

Require: Reference model π_{ref} , prompt distribution $\mathcal{D}$, number of completions per prompt G , number of inner updates μ , prompt token masking probability p_{mask}

```
1: Initialize  $\pi_\theta \leftarrow \pi_{\text{ref}}$ 
2: while not converged do
3:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ 
4:   Sample a prompt  $q \sim \mathcal{D}$ 
5:   Sample  $G$  completions  $o_i \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ ,  $i \in [G]$ 
6:   For each  $o_i$ , compute reward  $r_i$  and advantage  $A_i^k(\pi_{\theta_{\text{old}}})$  using Equation (2)
7:   for gradient update iterations  $n = 1, \dots, \mu$  do
8:      $q' \leftarrow$  randomly mask tokens of prompt  $p$  with probability  $p_{\text{mask}}$ 
9:     For  $\pi_\theta, \pi_{\theta_{\text{old}}}, \pi_{\text{ref}}$ , estimate log-probabilities of  $o_i$  given  $q'$  according to Section 3.1
10:    Compute diffu-GRPO objective (4) and update  $\pi_\theta$  by gradient descent
11: return  $\pi_\theta$ 
```

Let $\phi^{\pi_\theta}(o^k | q')$ and $\phi^{\pi_\theta}(o | q')$ denote the estimated per-token and sequence probabilities for π_θ . We derive the loss function of *diffu*-GRPO,

$$\mathcal{L}_{\text{diffu-GRPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, q' \sim \text{masking}(q), o_1, \dots, o_G \sim \pi_{\theta_{\text{old}}}(\cdot | q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{k=1}^{|o_i|} \min \left(\frac{\phi^{\pi_\theta}(o_i^k | q')}{\phi^{\pi_{\theta_{\text{old}}}}(o_i^k | q')} A_i^k, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\phi^{\pi_\theta}(o_i^k | q')}{\phi^{\pi_{\theta_{\text{old}}}}(o_i^k | q')}, 1 - \varepsilon, 1 + \varepsilon \right) A_i^k \right) - \beta D_{\text{KL}}[\phi^{\pi_\theta}(\cdot | q') \parallel \phi^{\pi_{\text{ref}}}(\cdot | q')] \right] \quad (4)$$

Our algorithm is summarized in Algorithm 1. To efficiently optimize the policy loss, in practice, on-policy RL algorithms such as PPO and GRPO perform multiple gradient updates for each batch of samples. During these updates, the prompt q , completions $\{o_i\}_{i=1}^G$, old policy $\pi_{\theta_{\text{old}}}$ and advantages $A_i^k(\pi_{\theta_{\text{old}}})$ are kept fixed. However, determining the optimal number of gradient updates per batch is challenging. If the number is too high, it can lead to overfitting within the batch, while a number that is too low slows down convergence. Achieving a balance between outer batch iterations and inner gradient updates is crucial for sample efficiency. Besides, every outer batch iteration requires sampling completion through iterative denoising steps, which incurs high computational cost.

Interestingly, our log-probability estimator offers a unique mitigation to this dilemma. For each gradient update step, we randomly mask the prompt q to q' to estimate the log-probabilities. Intuitively, this stochastic masking introduces perturbed views of the same (prompt, completion) pairs, serving as a form of regularization for policy optimization. It can also be viewed as a form of data augmentation, extracting more supervision signals from the same data. Empirically, we found that this approach, unique to masked diffusion models, allows us to scale μ to higher values while maintaining stable learning dynamics. As a consequence, it reduces the number of outer batch iterations required for convergence, which in turn decreases the number of online generations needed and ultimately results in significantly lower computational cost. As shown in Figure 5, training with higher values of μ achieves the same reward performance in substantially less wall clock time.

3.3 Supervised FineTuning with Reasoning Data

We perform SFT of LLaDA on s1K [28], a curated dataset consisting of 1000 high-quality reasoning questions. The reasoning traces in s1K exhibit detailed step-by-step problem-solving processes, including verification of intermediate results and backtracking when encountering errors or dead ends. The SFT algorithm is summarized in Algorithm 2, where tokens are randomly masked during training according to a time-varying schedule. The model is optimized to predict the original tokens given their context. We find that for SFT to work effectively in practice, various design choices must be carefully considered, whose details are discussed in Appendix D.2.

4 Experiments

To understand how reasoning capabilities can be scaled in masked dLLMs through training adaptations, we conduct comprehensive experiments to answer the following main research questions:

Table 1: **Model performance on Mathematics and Planning Benchmarks:** **Green values** indicate best performance and **blue values** indicate second-best performance. The results demonstrate that **d1-LLaDA** consistently outperforms other models, applying *diffu*-GRPO consistently improves the starting checkpoint, and *diffu*-GRPO alone shows better performance than SFT.

Model / Seq Len	GSM8K			MATH500			Countdown			Sudoku		
	128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	68.7	76.7	78.2	26.0	32.4	36.2	20.7	19.5	16.0	11.7	6.7	5.5
+ SFT	66.5	78.8	81.1	26.2	32.6	34.8	20.3	14.5	23.8	16.5	8.5	4.6
+ <i>diffu</i> -GRPO	72.6	79.8	81.9	33.2	37.2	39.2	33.2	31.3	37.1	18.4	12.9	11.0
+ SFT + <i>diffu</i> -GRPO (d1-LLaDA)	73.2	81.1	82.1	33.8	38.6	40.2	34.8	32.0	42.2	22.1	16.7	9.5

- (1) How do SFT on reasoning traces and applying *diffu*-GRPO *independently* improve LLaDA’s reasoning capabilities?
- (2) What additional gains can be achieved by *combining* SFT and *diffu*-GRPO to create d1-LLaDA?
- (3) **Design Choices:** How does the proposed log-probability estimation with *randomized masking* in *diffu*-GRPO and the masking probability p_{mask} affect training efficiency and stability?

4.1 Models, Tasks and Setups

Models We employ LLaDA-8B-Instruct [30], a state-of-the-art open-sourced dLLM that has not undergone post-training, as our primary experimental testbed and baseline. We apply 3 post-training recipes to LLaDA-8B-Instruct: (a) SFT, (b) *diffu*-GRPO, (c) d1: applying *diffu*-GRPO on the checkpoint after SFT, where we refer to them as LLaDA+SFT, LLaDA+*diffu*-GRPO, and d1-LLaDA, respectively.

Tasks We conduct experiments on six reasoning tasks in three categories: (1) **Mathematical reasoning:** we use GSM8K [10], a dataset of multi-step grade school math problems, and MATH500 [23], a curated subset of 500 problems drawn from the MATH dataset [18] comprising high-school competition math problems; (2) **Planning:** this includes two tasks: 4x4 Sudoku puzzles, which require constraint satisfaction and systematic elimination to fill a grid with numbers; and Countdown with 3 numbers, a combinatorial arithmetic game in which models must reach target numbers using basic arithmetic operations on a given set of numbers. (3) **Coding:** comprises of two benchmarks; HumanEval [8], a suite of 164 hand-crafted Python algorithmic programming problems and MBPP [6], a crowd-sourced collection of 257 Python tasks.

Training For SFT, we train on s1k [28] for 20 epochs, with a sequence length of 4096. For RL, we train a separate model for each task. More specifically, for GSM8K, MATH500, we train on the training split; for Countdown and Sudoku, we train on synthetic generated datasets. We use a composed reward function that combines both formatting and correctness rewards. Due to the heavy computational cost of online generations, we limit the generation sequence length of online generations to be 256 throughout RL training. Other hyperparameters of training, training and evaluation datasets, reward functions, and inference setups are detailed in Appendix D.

Evaluation For all the benchmarks, we evaluate LLaDA-8B-Instruct and LLaDA+SFT on the final checkpoint for all the tasks. For LLaDA+*diffu*-GRPO and d1-LLaDA, we evaluate every 100 steps starting from step 600 and report the best results. We evaluate all models with 0-shot-prompting and greedy decoding with generation lengths of 128, 256 and 512 separately.

4.2 Main Results

***diffu*-GRPO outperforms both LLaDA and SFT and improves over initialization checkpoint consistently .** Table 1 reports the performance of baseline LLaDA-8B-Instruct and models obtained by different post-training recipes across four tasks using zero-shot evaluation, where each *diffu*-GRPO model was trained for each task. For each task, we evaluate with three generation sequence lengths, and Figure 4 plots the average number of effective tokens. We present the following predominant findings.

Both *diffu*-GRPO and SFT yield improvements over the LLaDA-8B-Instruct baseline, with *diffu*-GRPO demonstrating consistently larger gains. Specifically, *diffu*-GRPO outperforms both LLaDA-8B-Instruct and SFT, in all 12 setups, while SFT outperforms LLaDA-8B-Instruct in only 7 of